



Guide 8: Project Persistence — Complete PRD

"Idea Library + Smart Recommendations"

Document:	Project Persistence — Feature PRD v1.0
Feature:	Cross-Cutting — All Modes Use This
Date:	June 2026
Status:	Final — Ready for Development

1. Feature Overview

Project Persistence system user ke sare sessions, ideas, blueprints, aur evolution history ko store karta hai. Ye sirf 'save' nahi — smart library hai with vector search, similarity recommendations, aur evolution lineage tracking.

Key Features: Save projects with full history, Revisit and evolve past ideas, Find similar ideas from history, Track evolution lineage (idea ka 'family tree'), Export entire project as ZIP, Share project links with team.

2. Complete Database Schema

```
model Project { id String @id @default(uuid()) userId String user User
@relation(fields: [userId], references: [id]) // Metadata title String description
String? mode Mode // GENERATOR | EVOLUTION | ARCHAEOLOGY status Status // DRAFT |
SELECTED | COMPLETED | ARCHIVED // Input originalInput String // Problem text /
Idea text / Code input // Generated content ideas Idea[] selectedIdeaId String?
selectedIdea Idea? @relation("SelectedIdea", fields: [selectedIdeaId], references:
[id]) // Deep dive (lazy-loaded) blueprint Blueprint? uiMockup UiMockup? //
Mode-specific data evolutionTree Json? // For EVOLUTION mode archaeologyReport
Json? // For ARCHAEOLOGY mode // Vector embedding for similarity search embedding
Unsupported("vector(1536)") // Timestamps createdAt DateTime @default(now())
updatedAt DateTime @updatedAt // Relations parentProjectId String? // If evolved
from another project childProjects Project[] @relation("ProjectEvolution")
parentProject Project? @relation("ProjectEvolution", fields: [parentProjectId],
references: [id]) } model Idea { id String @id @default(uuid()) projectId String
project Project @relation(fields: [projectId], references: [id]) title String
description String problemStatement String solution String keyFeatures String[] //
Scores innovationScore Int feasibilityScore Int difficulty String // EASY | MEDIUM
| HARD // Tags for filtering tags String[] platform String // WEB | MOBILE | AI_ML |
etc. // Selection selectedBy Project? @relation("SelectedIdea") // Deep dive
(lazy-loaded) blueprint Blueprint? uiMockup UiMockup? createdAt DateTime
@default(now()) } model Blueprint { id String @id @default(uuid()) ideaId String
@unique idea Idea @relation(fields: [ideaId], references: [id]) projectId String
@unique project Project @relation(fields: [projectId], references: [id]) // 7
sections overview String prd Json techStack Json databaseSchema Json apiDesign Json
userFlow Json implementationPlan Json // Export formats markdown String openApiSpec
String? mermaidDiagrams Json? createdAt DateTime @default(now()) } model UiMockup {
id String @id @default(uuid()) ideaId String @unique idea Idea @relation(fields:
[ideaId], references: [id]) projectId String @unique project Project
@relation(fields: [projectId], references: [id]) html String css String components
Json styleVariant String createdAt DateTime @default(now()) } enum Mode {
GENERATOR, EVOLUTION, ARCHAEOLOGY } enum Status { DRAFT, SELECTED, COMPLETED,
ARCHIVED }
```

3. Vector Search & Similarity Recommendations

Har project ka OpenAI embedding store karte hain. New input ke saath semantic similarity search karte hain. User ko 'Similar to your past project X...' recommendations milti hain.

```
// Vector Search Implementation async function findSimilarProjects(userId,
newInput, topK = 5) { // 1. Embed new input const embedding = await
openai.embeddings.create({ input: newInput, model: "text-embedding-3-small" }); //
2. Search in user's projects const similarProjects = await prisma.$queryRaw` SELECT
p.id, p.title, p.mode, p.createdAt, 1 - (p.embedding <=>
${embedding.data[0].embedding}::vector) as similarity FROM "Project" p WHERE
p.userId = ${userId} AND p.status != 'ARCHIVED' ORDER BY similarity DESC LIMIT
${topK} `; // 3. Filter high similarity only return similarProjects.filter(p =>
p.similarity > 0.75); } // Usage: When user enters new problem const
recommendations = await findSimilarProjects(userId, "Students struggle to find
internships"); // Returns: [{id: "proj_1", title: "Job Finder Dashboard",
similarity: 0.82}, ...]
```

4. Evolution Lineage Tracking

Har project ka 'family tree' track karte hain. Idea A se evolve karke Idea B bana, phir Idea B se Idea C — ye lineage database mein parent-child relation se store hoti hai. User dekh sakta hai apni idea kaise evolve hui.

```
// Evolution Lineage Example Project 1: "Smart Attendance System" (GENERATOR) ■■■■  
Selected: "AI-Powered Predictive Attendance" ■■■■ Evolved to → Project 2:  
"Predictive Attendance AI" (EVOLUTION) ■■■■ Selected: "Multi-Campus Predictive  
Attendance" ■■■■ Evolved to → Project 3: "Enterprise Attendance Platform"  
(EVOLUTION) // Database Relations Project 1: parentProjectId = null (root) Project  
2: parentProjectId = Project 1.id Project 3: parentProjectId = Project 2.id //  
Query lineage async function getLineage(projectId) { const lineage = []; let  
current = await prisma.project.findUnique({ where: { id: projectId } }); // Go up  
to root while (current.parentProjectId) { lineage.unshift(current); current = await  
prisma.project.findUnique({ where: { id: current.parentProjectId } }); }  
lineage.unshift(current); // Add root return lineage; // [Project 1, Project 2,  
Project 3] }
```

5. User Dashboard Design

Dashboard mein user ke sare projects organized hain. Filters: By mode (Generate/Evolve/Analyze), By status (Draft/Selected/Completed), By date, By platform, By tags. Search bar with semantic search. Quick actions: Continue, Evolve, Export, Delete, Share.

Recent Projects

Last 5 projects with quick actions. Cards show: title, mode badge, status, created date, overall score.

Evolution Timeline

Visual timeline of all evolution chains. Click kisi bhi node pe → full project open.

Similar Ideas

'You might also like...' section based on vector search. Shows similarity score.

Stats Overview

Total projects, ideas generated, evolutions done, average score, favorite platform.

Quick Start

Templates: 'New Problem', 'Evolve Existing', 'Analyze Code' — one-click start.

6. Export & Sharing Features

Export as Markdown

Complete project documentation: problem, ideas, selected idea, blueprint, all sections. GitHub README ready.

Export as PDF

Styled PDF with logo, cover page, table of contents, all sections. Stakeholder presentation ready.

Export as ZIP

HTML mockup + Markdown blueprint + JSON data. Complete project package for development.

Share Link

Public/private shareable link. Viewer can see project but not edit. Team collaboration ke liye.

Notion Export

Direct Notion page create with sections as blocks. Future integration.

7. Interview Talking Points

Why Vector Search for Project History?

Keyword search mein 'internship' search karne pe sirf 'internship' wale projects aayenge. Semantic search mein 'students struggle to find internships' se 'job matching', 'career platform', 'skill gap analyzer' bhi match honge. User ko pata bhi nahi hota ki usne similar idea pehle socha tha.

How Does Evolution Lineage Help?

User dekh sakta hai: 'Mera original idea Smart Attendance tha, phir AI version bana, phir Enterprise version.' Ye 'idea journey' hai. Portfolio mein dikhane ke liye perfect — shows iterative thinking.

What About Data Privacy?

User ka data isolated hai — sirf uske apne projects searchable hain. Vector embeddings bhi user-specific hain. No cross-user data sharing. GDPR compliant: user can delete all data anytime.

How Do You Handle Storage Costs?

Lazy loading se heavy assets (blueprints, mockups) sirf selected ideas ke liye store hote hain. Unselected ideas lightweight text cards hain. Old projects auto-archive ho jaate hain after 6 months of inactivity.

— End of Guide 8: Project Persistence PRD —